

Package ‘MVCens’

October 8, 2026

Title Matrix-Variate Models for Censored, Skewed, and Heavy-Tailed Data

Version 0.1.0.9000

Author Átila Prates Correia [aut, cre],
Victor Hugo Lachos [aut],
Carlos Alberto Ribeiro Diniz [aut]

Maintainer Átila Prates Correia <atilapc1989@gmail.com>

Description Provides simulation and likelihood-based estimation procedures for matrix-variate statistical models with emphasis on censored, incomplete, skewed, and heavy-tailed data. The package implements unified interfaces for random generation and ECM-based maximum likelihood estimation of matrix-variate normal, skew-normal, row skew-normal, skew-t, normal-inverse Gaussian, variance-gamma, and censored matrix-variate models. It is intended for applications involving matrix-valued observations such as longitudinal studies, spatio-temporal data, environmental monitoring, financial panels, and other structured multivariate datasets.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Imports LaplacesDemon,

Matrix,
matrixNormal,
mnormt,
MomTrunc,
mvtnorm,
statmod,
tlrmvnmvt,
tmvtnorm,
dplyr,
tidyr,
readr,
purrr,
tibble

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Contents

MVCens-package	2
dmvren	3
dmvrsn	5
dmvsn	6
dmvst	7
loglik_mvn	8
loglik_mvnc	9
loglik_mvren	10
loglik_mvrsn	11
loglik_mvsn	12
loglik_mvsnrc	13
loglik_mvst	14
mvren_article_parameters	15
mvren_covariances	16
mvren_mean	17
mvrsn_article_parameters	18
mvrsn_covariances	19
mvrsn_mean	20
mv_fit	20
mv_monte_carlo	25
mv_random	30
prepare_beijing_mvren	34
prepare_dj_data	36
Index	38

MVCens-package

MVCens: Matrix-Variate Models with Censoring and Asymmetry

Description

MVCens provides a common interface for generating, fitting, and evaluating matrix-variate normal, censored, skewed, and heavy-tailed models. The main user workflow is built around [mv_random\(\)](#), [mv_fit\(\)](#), and [mv_monte_carlo\(\)](#). Lower-level density, log-likelihood, moment, and reference-parameter functions are also available for direct calculations.

Main workflow

- Use [mv_random\(\)](#) to generate matrix-valued observations.
- Use [mv_fit\(\)](#) to estimate a registered model by ECM.
- Use [mv_monte_carlo\(\)](#) to assess an estimator in repeated samples.

Supported models

The high-level API recognizes MVN, MVNC, MVSN, MVSNC, MVST, MVRSN, and MVREN. Generation is additionally available for MVNIG and MVVG. Censored models use `cc` as the censoring indicator and `LS` as the array of censoring limits.

Numerical convention

Matrix observations are stored in arrays of dimensions p by q by n . Covariance structures use a p by p row component (`Sigma`) and a q by q column component (`Psi`). Consult each function's help page for its model-specific arguments and return value.

Author(s)

Maintainer: Átila Prates Correia <atilapc1989@gmail.com>

Authors:

- Átila Prates Correia <atilapc1989@gmail.com>
- Victor Hugo Lachos <hlachos@uconn.edu>
- Carlos Alberto Ribeiro Diniz <dcad@ufscar.br>

See Also

[mv_random\(\)](#), [mv_fit\(\)](#), [mv_monte_carlo\(\)](#)

dmvren

Density of the matrix-variate row exponential-normal distribution

Description

Computes the density or log-density of the matrix-variate row exponential-normal (MVREN) distribution at one p by q matrix-valued observation. MVREN replaces a common latent shift with independent row-specific exponential variables, so each row can have its own asymmetric mean-shift magnitude while the Gaussian component keeps separable row and column covariance. The MVREN model extends the matrix-variate normal distribution by introducing independent, nonnegative row-specific latent effects, allowing different rows of the observation matrix to exhibit distinct directions and degrees of asymmetry.

Usage

```
dmvren(  
  X,  
  M,  
  A,  
  Sigma,  
  Psi,  
  lambda = NULL,
```

```

log = FALSE,
eig_floor = 1e-10,
q_policy = c("warn", "strict", "regularize")
)

```

Arguments

<code>X</code>	A numeric p by q observation matrix.
<code>M</code>	A numeric p by q location matrix.
<code>A</code>	A numeric p by q row-specific latent-effect matrix.
<code>Sigma</code>	A symmetric positive-definite p by p row covariance matrix.
<code>Psi</code>	A symmetric positive-definite q by q column covariance matrix.
<code>lambda</code>	An optional numeric vector of length p containing finite, strictly positive exponential rate parameters. When NULL, a vector of ones is used. The high-level MVREN API fixes this vector to ones; non-unit values here are supported only to evaluate a supplied general parameterization.
<code>log</code>	A logical scalar indicating whether the log-density should be returned. The default is FALSE.
<code>eig_floor</code>	A positive numeric scalar specifying the minimum numerical eigenvalue allowed for the matrix Q. The default is 1e-10.
<code>q_policy</code>	Character string indicating how to handle cases in which the smallest eigenvalue of Q is less than eig_floor. Available options are "warn", which regularizes Q and issues a warning; "strict", which stops with an error; and "regularize", which applies the regularization silently.

Details

The identified MVREN model used by `mv_random()` and `mv_fit()` fixes every `lambda_i` to one. This lower-level density evaluator retains the optional rate vector so that a supplied general parameterization can be evaluated explicitly.

Value

A numeric scalar containing the MVREN density evaluated at X. When `log = TRUE`, the corresponding log-density is returned.

See Also

Other MVCens density functions: [dmvrsn\(\)](#), [dmvsn\(\)](#), [dmvst\(\)](#)

Other MVCens MVREN functions: [loglik_mvren\(\)](#), [mvren_article_parameters\(\)](#), [mvren_covariances\(\)](#), [mvren_mean\(\)](#)

Examples

```
X <- matrix(seq(-0.2, 0.9, length.out = 12), 3, 4)
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
dmvren(
  X = X, M = M, A = A,
  Sigma = diag(3), Psi = diag(4)
)
```

dmvrsn

Density of the matrix-variate row skew-normal distribution

Description

Evaluates the closed-form probability density or log-density of a matrix-variate row skew-normal distribution at one p by q matrix-valued observation. Unlike the common-latent MVSN model, MVRSN assigns an independent half-normal skewing variable to each row, allowing heterogeneous row-level asymmetry while retaining separable matrix-normal row and column covariance components.

Usage

```
dmvrsn(X, M, A, Sigma, Psi, log = FALSE)
```

Arguments

<code>X</code>	Matrix of dimension $p \times q$.
<code>M</code>	Location matrix of dimension $p \times q$.
<code>A</code>	Skewness matrix of dimension $p \times q$.
<code>Sigma</code>	Positive definite row covariance matrix of dimension $p \times p$.
<code>Psi</code>	Positive definite column covariance matrix of dimension $q \times q$.
<code>log</code>	Logical. If TRUE, returns the log-density.

Value

A numeric scalar containing the density evaluated at `X`. When `log = TRUE`, the corresponding log-density is returned.

See Also

Other MVCens density functions: [dmvren\(\)](#), [dmvsn\(\)](#), [dmvst\(\)](#)

Other MVCens MVRSN functions: [loglik_mvrsn\(\)](#), [mvrsn_article_parameters\(\)](#), [mvrsn_covariances\(\)](#), [mvrsn_mean\(\)](#)

Examples

```
X <- matrix(seq(-0.2, 0.9, length.out = 12), 3, 4)
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
dmvrsn(
  X = X, M = M, A = A,
  Sigma = diag(3), Psi = diag(4)
)
```

dmvsn

Density of the multivariate skew-normal distribution

Description

Evaluates the probability density of the multivariate skew-normal kernel used by the matrix-variate skew-normal model. The density corresponds to a Gaussian component perturbed by a half-normal latent skewing variable. The matrix-variate likelihood functions call this kernel after vectorizing each p by q observation.

Usage

```
dmvsn(y, mu, Sigma, lambda, epsilon = 1e-08)
```

Arguments

<code>y</code>	Numeric vector containing the values at which the density is evaluated.
<code>mu</code>	Numeric vector specifying the location parameter. It must have the same length as <code>y</code> .
<code>Sigma</code>	Numeric positive definite covariance matrix associated with the symmetric component of the distribution. Its number of rows and columns must equal the length of <code>mu</code> .
<code>lambda</code>	Numeric vector specifying the direction and magnitude of skewness. It must have the same length as <code>mu</code> .
<code>epsilon</code>	Positive numeric scalar specifying the tolerance used for covariance-matrix regularization and numerical positive-definiteness checks. The default is $1e-8$.

Value

A numeric scalar containing the density evaluated at `y`.

See Also

Other MVCens density functions: [dmvren\(\)](#), [dmvrsn\(\)](#), [dmvst\(\)](#)

Examples

```
Y <- matrix(seq(-0.2, 0.9, length.out = 12), 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
dmvsn(as.vector(Y), mu = rep(0, 12), Sigma = diag(12), lambda = as.vector(A))
```

dmvst

Density of the multivariate skew- t distribution

Description

Evaluates the probability density or log-density of the multivariate skew- t kernel used by the matrix-variate skew- t model. The distribution combines a skew-normal component with a Gamma scale mixture, allowing both asymmetry and heavy tails. Matrix-variate skew- t observations are handled by [loglik_mvst\(\)](#), which vectorizes each p by q slice before calling this density.

Usage

```
dmvst(y, mu, Sigma, lambda, nu, epsilon = 1e-08, log = FALSE)
```

Arguments

<code>y</code>	Numeric vector of observed values.
<code>mu</code>	Numeric location vector with the same length as <code>y</code> .
<code>Sigma</code>	Positive definite scale matrix with dimensions <code>length(y)</code> by <code>length(y)</code> .
<code>lambda</code>	Numeric skewness vector with the same length as <code>y</code> .
<code>nu</code>	Positive scalar. Degrees of freedom.
<code>epsilon</code>	Numerical tolerance for positive-definiteness corrections.
<code>log</code>	Logical. If TRUE, returns the log-density.

Value

A numeric scalar containing the density, or the log-density when `log = TRUE`.

See Also

Other MVCens density functions: [dmvren\(\)](#), [dmvrnsn\(\)](#), [dmvsn\(\)](#)

Examples

```
Y <- matrix(seq(-0.2, 0.9, length.out = 12), 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
dmvst(
  y = as.vector(Y), mu = rep(0, 12), Sigma = diag(12),
  lambda = as.vector(A), nu = 5
)
```

loglik_mvn	<i>Matrix-variate normal log-likelihood</i>
------------	---

Description

Computes the total observed-data log-likelihood under the matrix-variate normal distribution for complete matrix-valued observations. Each p by q slice in `samples` is evaluated using the location matrix `M`, row covariance `Sigma`, and column covariance `Psi`; the individual log-density contributions are summed.

Usage

```
loglik_mvn(samples, M, Sigma, Psi)
```

Arguments

<code>samples</code>	Numeric array with dimensions $p \times q \times n$.
<code>M</code>	Location matrix of dimension $p \times q$.
<code>Sigma</code>	Row covariance matrix of dimension $p \times p$.
<code>Psi</code>	Column covariance matrix of dimension $q \times q$.

Value

A finite numeric scalar containing the total log-likelihood.

See Also

Other MVCens likelihood functions: [loglik_mvnc\(\)](#), [loglik_mvren\(\)](#), [loglik_mvrsn\(\)](#), [loglik_mvsn\(\)](#), [loglik_mvsnc\(\)](#), [loglik_mvst\(\)](#)

Examples

```
M <- matrix(0, 3, 4)
x <- array(seq(-0.2, 0.9, length.out = 12), dim = c(3, 4, 1))
loglik_mvn(x, M = M, Sigma = diag(3), Psi = diag(4))
```

loglik_mvnc	<i>Censored matrix-variate normal log-likelihood</i>
-------------	--

Description

Computes the observed-data log-likelihood for censored and/or missing matrix-variate normal data, following the matrix-normal censored-data formulation based on truncated moments. Fully observed matrices are evaluated through the matrix-normal density. Fully censored matrices are evaluated through the matrix-normal probability, and partially censored matrices are handled by conditioning on the observed entries.

Usage

```
loglik_mvnc(cc, LS, samples, M, Sigma, Psi, epsilon = 1e-08)
```

Arguments

cc	Censoring indicator array with the same dimensions as <code>samples</code> . Entries equal to 1 indicate censored or missing values; entries equal to 0 indicate observed values.
LS	Array of upper censoring limits with the same dimensions as <code>samples</code> . Censored entries must satisfy <code>samples <= LS</code> .
samples	Numeric array with dimensions $p \times q \times n$.
M	Location matrix of dimension $p \times q$.
Sigma	Row covariance matrix of dimension $p \times p$.
Psi	Column covariance matrix of dimension $q \times q$.
epsilon	Positive tolerance used for numerical stabilization.

Value

A finite numeric scalar containing the censored-data log-likelihood.

See Also

Other MVCens likelihood functions: [loglik_mvnc\(\)](#), [loglik_mvren\(\)](#), [loglik_mvrsn\(\)](#), [loglik_mvsn\(\)](#), [loglik_mvsnrc\(\)](#), [loglik_mvst\(\)](#)

Examples

```
M <- matrix(0, 3, 4)
x <- array(seq(-0.2, 2.1, length.out = 24), dim = c(3, 4, 2))
cc <- array(0, dim = dim(x))
cc[1, 1, 2] <- 1
limits <- array(Inf, dim = dim(x))
limits[1, 1, 2] <- 1.2
loglik_mvnc(cc, limits, x, M, diag(3), diag(4))
```

loglik_mvren

*Observed-data log-likelihood of the MVREN distribution***Description**

Evaluates the observed-data log-likelihood of a sample from the matrix-variate row exponential-normal (MVREN) distribution by summing the individual log-density contributions of the matrix-valued observations. Each slice $X_array[, , i]$ is evaluated by `dmvren()` under the row-specific exponential latent-effect representation.

Usage

```
loglik_mvren(
  X_array,
  M,
  A,
  Sigma,
  Psi,
  lambda = NULL,
  eig_floor = 1e-10,
  q_policy = c("warn", "strict", "regularize")
)
```

Arguments

<code>X_array</code>	A numeric three-dimensional array with dimensions p by q by n , containing n matrix-valued observations.
<code>M</code>	A numeric p by q location matrix.
<code>A</code>	A numeric p by q row-specific latent-effect matrix.
<code>Sigma</code>	A symmetric positive-definite p by p row covariance matrix.
<code>Psi</code>	A symmetric positive-definite q by q column covariance matrix.
<code>lambda</code>	An optional numeric vector of length p containing finite, strictly positive exponential rate parameters. When <code>NULL</code> , a vector of ones is used. The high-level MVREN API fixes this vector to ones; non-unit values here are supported only to evaluate a supplied general parameterization.
<code>eig_floor</code>	A positive numeric scalar specifying the minimum numerical eigenvalue allowed for the matrix Q used in each density evaluation. The default is $1e-10$.
<code>q_policy</code>	A character string specifying how to handle a matrix Q whose smallest eigenvalue is below <code>eig_floor</code> . Available options are "warn", "strict", and "regularize"; see <code>dmvren()</code> for details.

Details

The identified MVREN model used by `mv_fit()` fixes every `lambda_i` to one. This lower-level evaluator retains the optional rate vector for explicit likelihood calculations under a supplied general parameterization.

Value

A numeric scalar containing the observed-data log-likelihood of the supplied sample.

See Also

Other MVCens likelihood functions: [loglik_mvn\(\)](#), [loglik_mvnc\(\)](#), [loglik_mvrsn\(\)](#), [loglik_mvsn\(\)](#), [loglik_mvsnc\(\)](#), [loglik_mvst\(\)](#)

Other MVCens MVREN functions: [dmvren\(\)](#), [mvren_article_parameters\(\)](#), [mvren_covariances\(\)](#), [mvren_mean\(\)](#)

Examples

```
x <- array(seq(-0.2, 2.1, length.out = 24), dim = c(3, 4, 2))
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
loglik_mvren(
  x, M = M, A = A,
  Sigma = diag(3), Psi = diag(4)
)
```

loglik_mvrsn

Log-likelihood function of the MVRSN distribution

Description

Sums the closed-form MVRSN log-density over a sample array with dimensions p by q by n . Each slice $X_array[, , i]$ is treated as one matrix-valued observation with row-specific latent skewness.

Usage

```
loglik_mvrsn(X_array, M, A, Sigma, Psi)
```

Arguments

X_array	Numeric sample array with dimensions p by q by n .
M	Numeric p by q location matrix.
A	Numeric p by q skewness matrix.
Σ	Positive-definite p by p row covariance matrix.
Ψ	Positive-definite q by q column covariance matrix.

Value

A finite numeric scalar containing the observed-data log-likelihood.

See Also

Other MVCens likelihood functions: [loglik_mvn\(\)](#), [loglik_mvnc\(\)](#), [loglik_mvren\(\)](#), [loglik_mvsn\(\)](#), [loglik_mvsnrc\(\)](#), [loglik_mvst\(\)](#)

Other MVCens MVRSN functions: [dmvrsn\(\)](#), [mvrsn_article_parameters\(\)](#), [mvrsn_covariances\(\)](#), [mvrsn_mean\(\)](#)

Examples

```
x <- array(seq(-0.2, 2.1, length.out = 24), dim = c(3, 4, 2))
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
loglik_mvrsn(
  x, M = M, A = A,
  Sigma = diag(3), Psi = diag(4)
)
```

loglik_mvsn

*Matrix-variate skew-normal log-likelihood***Description**

Computes the total log-likelihood for complete matrix-variate skew-normal data. The model uses the stochastic representation $X = M + W A + V$, where W is a positive half-normal latent variable and V is matrix-normal noise with separable row and column covariance. Each p by q observation is vectorized and evaluated through the corresponding multivariate skew-normal density with Kronecker covariance structure.

Usage

```
loglik_mvsn(X, muM, AM, SigmaM, PsiM, epsilon = 1e-08)
```

Arguments

<code>X</code>	Numeric array with dimensions $p \times q \times n$.
<code>muM</code>	Location matrix of dimension $p \times q$.
<code>AM</code>	Skewness matrix of dimension $p \times q$.
<code>SigmaM</code>	Row covariance matrix of dimension $p \times p$.
<code>PsiM</code>	Column covariance matrix of dimension $q \times q$.
<code>epsilon</code>	Numerical tolerance for covariance regularization.

Value

A finite numeric scalar containing the total log-likelihood.

See Also

Other MVCens likelihood functions: [loglik_mvn\(\)](#), [loglik_mvnc\(\)](#), [loglik_mvren\(\)](#), [loglik_mvrsn\(\)](#), [loglik_mvsnrc\(\)](#), [loglik_mvst\(\)](#)

Examples

```
x <- array(seq(-0.2, 2.1, length.out = 24), dim = c(3, 4, 2))
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
loglik_mvsn(
  x, muM = matrix(0, 3, 4), AM = A,
  SigmaM = diag(3), PsiM = diag(4)
)
```

loglik_mvsnrc

*Censored matrix-variate skew-normal log-likelihood***Description**

Computes the observed-data log-likelihood for censored and/or missing matrix-variate skew-normal data. The model extends the censored matrix-normal likelihood by adding the half-normal latent skewness variable used in the MVSN representation. Fully observed observations are evaluated by the extended skew-normal density, fully censored observations by the corresponding probability, and partially censored observations by conditional decomposition.

Usage

```
loglik_mvsnrc(cc, LS, X, muM, SigmaM, PsiM, lambdaM, epsilon = 1e-08)
```

Arguments

cc	Censoring indicator array with the same dimensions as X. Entries equal to 1 indicate censored or missing values; entries equal to 0 indicate observed values.
LS	Array of upper censoring limits with the same dimensions as X. Censored entries must satisfy $X \leq LS$.
X	Numeric array with dimensions $p \times q \times n$.
muM	Location matrix of dimension $p \times q$.
SigmaM	Row covariance matrix of dimension $p \times p$.
PsiM	Column covariance matrix of dimension $q \times q$.
lambdaM	Skewness matrix of dimension $p \times q$.
epsilon	Numerical tolerance for covariance regularization.

Value

A finite numeric scalar containing the observed-data log-likelihood.

See Also

Other MVCens likelihood functions: [loglik_mvn\(\)](#), [loglik_mvnc\(\)](#), [loglik_mvren\(\)](#), [loglik_mvrsn\(\)](#), [loglik_mvsn\(\)](#), [loglik_mvst\(\)](#)

Examples

```
x <- array(seq(-0.2, 2.1, length.out = 24), dim = c(3, 4, 2))
cc <- array(0, dim = dim(x))
cc[1, 1, 2] <- 1
limits <- array(Inf, dim = dim(x))
limits[1, 1, 2] <- 1.2
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
loglik_mvsn(
  cc, limits, x, muM = matrix(0, 3, 4), SigmaM = diag(3),
  PsiM = diag(4), lambdaM = A
)
```

loglik_mvst

*Matrix-variate skew-t log-likelihood***Description**

Computes the log-likelihood for complete matrix-variate skew- t data. The MVST construction extends the skew-normal matrix model with a Gamma scale mixture, so that A controls asymmetry while ν controls tail weight. Each p by q observation in X is vectorized, using the Kronecker row and column scale structure implied by Σ_M and Ψ_M , and evaluated by [dmvst\(\)](#).

Usage

```
loglik_mvst(nu, X, muM, AM, SigmaM, PsiM, epsilon = 1e-08)
```

Arguments

<code>nu</code>	Positive scalar. Degrees of freedom.
<code>X</code>	Numeric array with dimensions $p \times q \times n$.
<code>muM</code>	Location matrix of dimension $p \times q$.
<code>AM</code>	Skewness matrix of dimension $p \times q$.
<code>SigmaM</code>	Row scale matrix of dimension $p \times p$.
<code>PsiM</code>	Column scale matrix of dimension $q \times q$.
<code>epsilon</code>	Numerical tolerance for positive-definiteness corrections.

Value

A finite numeric scalar containing the total log-likelihood.

See Also

Other MVCens likelihood functions: [loglik_mvsn\(\)](#), [loglik_mvnc\(\)](#), [loglik_mvren\(\)](#), [loglik_mvrsn\(\)](#), [loglik_mvsn\(\)](#), [loglik_mvsnc\(\)](#)

Examples

```
x <- array(seq(-0.2, 2.1, length.out = 24), dim = c(3, 4, 2))
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
loglik_mvst(
  nu = 5, X = x, muM = matrix(0, 3, 4),
  AM = A, SigmaM = diag(3), PsiM = diag(4)
)
```

mvren_article_parameters

Predefined parameter set for the MVREN distribution

Description

Constructs a predefined collection of 3 by 4 parameters for the MVREN Monte Carlo setting described in the row exponential-normal article. The setting is intended for simulation studies, numerical experiments, and reproducible examples involving row-specific exponential latent effects.

Usage

```
mvren_article_parameters(normalize_Psi = TRUE)
```

Arguments

normalize_Psi	Logical scalar indicating whether Psi should be normalized to have determinant one. The corresponding reciprocal scaling is applied to Sigma so that the Kronecker-product covariance structure remains unchanged. The default is TRUE.
---------------	---

Details

The returned parameter set uses the identified MVREN convention $\lambda_{i,i} = 1$ for every row. The `normalize_Psi` option concerns only the separate row/column covariance scale constraint; rows of A are not normalized. The parameter values are provided as a reproducible reference set; they are not fitted or estimated by this function.

Value

A list containing the matrices M, A, Sigma, and Psi, together with the exponential-rate vector lambda.

See Also

Other MVCens MVREN functions: [dmvren\(\)](#), [loglik_mvren\(\)](#), [mvren_covariances\(\)](#), [mvren_mean\(\)](#)

Examples

```
pars <- mvren_article_parameters()
names(pars)
det(pars$Psi)
```

mvren_covariances	<i>Compute row and column covariance summaries of an MVREN distribution</i>
-------------------	---

Description

Computes the row and column covariance summaries associated with a matrix-variate row exponential-normal distribution from its latent-effect matrix, row and column covariance matrices, and row-specific exponential rate parameters. These summaries combine the separable matrix-normal covariance contribution with the variability induced by independent exponential row effects. They describe row-wise and column-wise second moments of the matrix-valued observation, not a fitted model.

Usage

```
mvren_covariances(M, A, Sigma, Psi, lambda = NULL)
```

Arguments

M	A numeric p by q location matrix. The covariance summaries do not depend on M ; it is used to establish and validate the model dimensions.
A	A numeric p by q latent-effect matrix.
Sigma	A symmetric positive-definite p by p row covariance matrix.
Psi	A symmetric positive-definite q by q column covariance matrix.
lambda	An optional numeric vector of length p containing finite, strictly positive exponential rate parameters. When <code>NULL</code> , a vector of ones is used. The high-level MVREN API fixes this vector to ones; non-unit values here are for evaluating a supplied general parameterization.

Details

For the identified model used by `mv_random()` and `mv_fit()`, use the default unit-rate vector. The optional argument is retained here to evaluate covariance summaries for an explicitly supplied general parameterization.

Value

A list containing:

row A $p \times p$ matrix equal to $\mathbb{E}[(\mathbf{X} - \mathbb{E}(\mathbf{X}))(\mathbf{X} - \mathbb{E}(\mathbf{X}))^\top]$.
column A $q \times q$ matrix equal to $\mathbb{E}[(\mathbf{X} - \mathbb{E}(\mathbf{X}))^\top(\mathbf{X} - \mathbb{E}(\mathbf{X}))]$.

See Also

Other MVCens MVREN functions: [dmvren\(\)](#), [loglik_mvren\(\)](#), [mvren_article_parameters\(\)](#), [mvren_mean\(\)](#)

Examples

```
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
mvren_covariances(
  M = M, A = A,
  Sigma = diag(3), Psi = diag(4)
)
```

mvren_mean

*Compute the mean matrix of an MVREN distribution***Description**

Computes the mean matrix of a matrix-variate row exponential-normal distribution from its location matrix, latent-effect matrix, and row-specific exponential rate parameters. In this model, independent exponential latent variables produce row-specific mean shifts through $X = M + W A + V$, where W is diagonal and V is matrix-normal noise.

Usage

```
mvren_mean(M, A, lambda = NULL)
```

Arguments

M	A numeric p by q location matrix.
A	A numeric p by q latent-effect matrix.
lambda	An optional numeric vector of length p containing finite, strictly positive exponential rate parameters. When NULL, a vector of ones is used. The high-level MVREN API (mv_random() and mv_fit()) fixes this vector to ones for identifiability; non-unit values here are supported only to evaluate a supplied general parameterization.

Details

For the identified model used by [mv_random\(\)](#) and [mv_fit\(\)](#), use the default unit-rate vector. The optional argument is retained here to evaluate moments for an explicitly supplied general parameterization.

Value

A numeric p by q matrix containing the mean of the MVREN distribution.

See Also

Other MVCens MVREN functions: [dmvren\(\)](#), [loglik_mvren\(\)](#), [mvren_article_parameters\(\)](#), [mvren_covariances\(\)](#)

Examples

```
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
mvren_mean(M, A)
```

```
mvrnsn_article_parameters
```

Parameter set used in the paper's Monte Carlo section

Description

Constructs a predefined collection of 3 by 4 parameters for the MVRSN Monte Carlo setting described in the row skew-normal article. The setting is designed to study recovery of M, A, Sigma, and Psi when asymmetry is driven by independent row-specific half-normal latent variables. Psi is normalized to determinant one by default, matching the row/column scale identifiability convention used by the implementation.

Usage

```
mvrnsn_article_parameters(normalize_Psi = TRUE)
```

Arguments

`normalize_Psi` Logical scalar indicating whether Psi is normalized to determinant one, with reciprocal scaling applied to Sigma.

Details

The parameter values are provided as a reproducible reference set; they are not fitted or estimated by this function.

Value

A named list containing M, A, Sigma, and Psi.

See Also

Other MVCens MVRSN functions: [dmvrnsn\(\)](#), [loglik_mvrnsn\(\)](#), [mvrnsn_covariances\(\)](#), [mvrnsn_mean\(\)](#)

Examples

```
pars <- mvrnsn_article_parameters()
names(pars)
det(pars$Psi)
```

mvrsn_covariances	<i>Row and column covariance summaries of the MVRSN distribution</i>
-------------------	--

Description

Computes the theoretical row and column covariance summaries of the matrix-variate row skew-normal distribution. The summaries combine the separable matrix-normal covariance contribution with the row-specific half-normal skewing contribution induced by A. They correspond to row-wise and column-wise second moments of the matrix-valued observation, not to the full vectorized Kronecker covariance matrix.

Usage

```
mvrsn_covariances(M, A, Sigma, Psi)
```

Arguments

M	Location matrix. It is accepted for API symmetry and dimension validation, but it does not enter the covariance formula.
A	Skewness matrix of dimension $p \times q$.
Sigma	Positive definite row covariance matrix of dimension $p \times p$.
Psi	Positive definite column covariance matrix of dimension $q \times q$.

Value

A list with row (p by p) and column (q by q) covariance summaries.

See Also

Other MVCens MVRSN functions: [dmvrsn\(\)](#), [loglik_mvrsn\(\)](#), [mvrsn_article_parameters\(\)](#), [mvrsn_mean\(\)](#)

Examples

```
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
mvrsn_covariances(
  M = M, A = A,
  Sigma = diag(3), Psi = diag(4)
)
```

mvrsn_mean	<i>Mean matrix of the MVRSN distribution</i>
------------	--

Description

Computes the theoretical mean under the matrix-variate row skew-normal (MVRSN) representation. In this model, independent standard half-normal latent variables act row by row through a diagonal skewing mechanism, producing the stochastic form $X = M + W A + V$, with matrix-normal noise V .

Usage

```
mvrsn_mean(M, A)
```

Arguments

M	Numeric p by q location matrix.
A	Numeric p by q skewness matrix.

Value

A numeric p by q theoretical mean matrix.

See Also

Other MVCens MVRSN functions: [dmvrsn\(\)](#), [loglik_mvrsn\(\)](#), [mvrsn_article_parameters\(\)](#), [mvrsn_covariances\(\)](#)

Examples

```
M <- matrix(0, 3, 4)
A <- matrix(seq(0.1, 1.2, length.out = 12), 3, 4)
mvrsn_mean(M, A)
```

mv_fit	<i>Fit a registered matrix-variate model by ECM</i>
--------	---

Description

Validates the common fit contract and dispatches to the model-specific EM-type implementation described by the corresponding model article. X must contain n matrix observations stored as a p by q by n array. Matrix-normal and skew-normal censored fits use ECM updates built around truncated moments, MVST uses an ECME-style heavy-tailed skew model, and MVRSN/MVREN use row-specific latent-variable ECM kernels. The model name is case-insensitive and the scientific update equations remain in the model kernel.

Usage

```

mv_fit(
  model,
  X = NULL,
  cc = NULL,
  LS = NULL,
  precision = 1e-06,
  max_iter = 500L,
  samples = NULL,
  epsilon = NULL,
  nu = NULL,
  get.nu = NULL,
  nu_bounds = NULL,
  normalize_Psi = NULL,
  M_init = NULL,
  A_init = NULL,
  Sigma_init = NULL,
  Psi_init = NULL,
  q_policy = NULL,
  verbose = NULL,
  eig_floor = NULL,
  monotone_tol = NULL,
  progress_callback = NULL
)

```

Arguments

model	One estimable model name. Matching is case-insensitive.
X	Numeric array of dimensions p by q by n, containing the matrix-valued observations.
cc	Optional censoring indicator array with the same dimensions as X; 1 marks censored/missing entries and 0 marks observed entries. Required for MVNC and MVSNC.
LS	Optional array of upper censoring limits, with the same dimensions as X. Required by MVNC and MVSNC together with cc.
precision	Positive finite ECM convergence tolerance.
max_iter	Positive integer maximum number of ECM iterations.
samples	Deprecated alias for X. If X = NULL and samples is supplied, samples is used and a deprecation warning is issued.
epsilon	Positive numerical tolerance used by MVSNC, MVSNC, and MVST. NULL uses the model-specific default.
nu	Degrees of freedom supplied to the MVST fit. NULL uses the model-specific default.
get.nu	Logical control for MVST; if supplied, determines whether the degrees of freedom are updated during fitting.

nu_bounds	Length-two numeric vector giving the admissible interval for the MVST degrees-of-freedom update. NULL uses the model default.
normalize_Psi	Logical control used by MVRSN to normalize Psi to determinant one while preserving the Kronecker covariance scale.
M_init	Optional initial location matrix for MVRSN and MVREN.
A_init	Optional initial latent-effect/skewness matrix for MVRSN and MVREN.
Sigma_init	Optional initial row covariance matrix for MVRSN and MVREN.
Psi_init	Optional initial column covariance matrix for MVRSN and MVREN.
q_policy	Character control for the MVREN Q-matrix handling policy; one of "warn", "strict", or "regularize". NULL uses the model default.
verbose	Optional logical verbosity control for MVRSN and MVREN. NULL uses the model-specific default.
eig_floor	Optional positive eigenvalue floor used by MVRSN. NULL uses the model-specific default.
monotone_tol	Optional tolerance for detecting decreases in the MVRSN likelihood path. NULL uses the model-specific default.
progress_callback	Optional function used internally by MVREN to report iteration progress. It should accept iteration, max_iter, and criterion.

Details

Models currently fitted by this interface are MVN, MVNC, MVSN, MVSNC, MVST, MVRSN, and MVREN. MVNIG and MVVG are generation-only models in this package and fail closed with an informative error when passed to `mv_fit()`.

ECM iterations are kept sequential because their update order is part of each model's scientific specification. The function is observational with respect to the input data: it returns a fit object and does not modify X, cc, or LS in place. Generation-only models fail closed instead of silently switching to another estimator. Model-specific controls are explicit arguments; controls that do not apply to the chosen model are ignored by the dispatcher.

Value

A model-specific fit object containing estimated parameters and, where implemented, the likelihood path, iteration count, convergence flag, BIC, and numerical diagnostics.

See Also

`mv_random()`, `mv_monte_carlo()`

Other MVCens main interface: `mv_monte_carlo()`, `mv_random()`

Examples

```
## The fits below use very few ECM iterations so that the examples are easy
## to experiment with. For real analyses, increase max_iter as needed.
```

```

set.seed(123)
p <- 2
q <- 2
n <- 30
M <- matrix(0, p, q)
A <- matrix(c(0.4, 0.2, 0.1, 0.3), p, q)
Sigma <- matrix(c(1.0, 0.2, 0.2, 1.0), p, p)
Psi <- matrix(c(1.0, 0.1, 0.1, 1.0), q, q)

## 1. MVN: complete matrix-normal data
x_mvn <- mv_random("MVN", n, M, Sigma = Sigma, Psi = Psi)
fit_mvn <- mv_fit(
  model = "MVN", X = x_mvn,
  precision = 1e-6, max_iter = 2
)
names(fit_mvn)

## Legacy compatibility: `samples` is a deprecated alias for `X`.
fit_mvn_legacy <- suppressWarnings(mv_fit(
  model = "MVN", samples = x_mvn,
  precision = 1e-6, max_iter = 1
))

## 2. MVNC: fit the observed/censored array together with cc and LS.
d_mvnc <- mv_random(
  "MVNC", n, M, Sigma = Sigma, Psi = Psi,
  cens = 0.15, Ind = 1
)
fit_mvnc <- mv_fit(
  model = "MVNC", X = d_mvnc$X.cens,
  cc = d_mvnc$cc, LS = d_mvnc$LS,
  precision = 1e-6, max_iter = 2
)

## The same fitting call applies to missing (Ind = 2) and mixed (Ind = 3)
## samples; only the generated cc/LS structures change.
d_mvnc_missing <- mv_random(
  "MVNC", n, M, Sigma = Sigma, Psi = Psi,
  cens = 0.15, Ind = 2
)
fit_mvnc_missing <- mv_fit(
  "MVNC", X = d_mvnc_missing$X.cens,
  cc = d_mvnc_missing$cc, LS = d_mvnc_missing$LS,
  max_iter = 1
)
d_mvnc_mixed <- mv_random(
  "MVNC", n, M, Sigma = Sigma, Psi = Psi,
  cens = 0.15, Ind = 3
)
fit_mvnc_mixed <- mv_fit(
  "MVNC", X = d_mvnc_mixed$X.cens,
  cc = d_mvnc_mixed$cc, LS = d_mvnc_mixed$LS,
  max_iter = 1
)

```

```

)

## 3. MVSNC: epsilon controls numerical truncation calculations.
x_mvsn <- mv_random(
  "MVSNC", n, M, A = A, Sigma = Sigma, Psi = Psi
)
fit_mvsn <- mv_fit(
  model = "MVSNC", X = x_mvsn,
  epsilon = 1e-8, precision = 1e-6, max_iter = 2
)

## 4. MVSNC: censored/missing skew-normal data.
d_mvsn <- mv_random(
  "MVSNC", n, M, A = A, Sigma = Sigma, Psi = Psi,
  cens = 0.15, Ind = 1
)
fit_mvsn <- mv_fit(
  model = "MVSNC", X = d_mvsn$X.cens,
  cc = d_mvsn$cc, LS = d_mvsn$LS,
  epsilon = 1e-8, precision = 1e-6, max_iter = 2
)
d_mvsn_missing <- mv_random(
  "MVSNC", n, M, A = A, Sigma = Sigma, Psi = Psi,
  cens = 0.15, Ind = 2
)
fit_mvsn_missing <- mv_fit(
  "MVSNC", X = d_mvsn_missing$X.cens,
  cc = d_mvsn_missing$cc, LS = d_mvsn_missing$LS,
  max_iter = 1
)
d_mvsn_mixed <- mv_random(
  "MVSNC", n, M, A = A, Sigma = Sigma, Psi = Psi,
  cens = 0.15, Ind = 3
)
fit_mvsn_mixed <- mv_fit(
  "MVSNC", X = d_mvsn_mixed$X.cens,
  cc = d_mvsn_mixed$cc, LS = d_mvsn_mixed$LS,
  max_iter = 1
)

## 5. MVST: keep nu fixed or update it during fitting.
x_mvst <- mv_random(
  "MVST", n, M, A = A, Sigma = Sigma, Psi = Psi, nu = 6
)
fit_mvst_fixed <- mv_fit(
  model = "MVST", X = x_mvst, nu = 6, get.nu = FALSE,
  epsilon = 1e-8, precision = 1e-6, max_iter = 2
)
fit_mvst_estimated <- mv_fit(
  model = "MVST", X = x_mvst, nu = 6, get.nu = TRUE,
  nu_bounds = c(2.01, 50), epsilon = 1e-8,
  precision = 1e-6, max_iter = 2
)

```

```

## 6. MVRSN: default normalization, or custom initialization/controls.
x_mvrsn <- mv_random(
  "MVRSN", n, M, A = A, Sigma = Sigma, Psi = Psi
)
fit_mvrsn <- mv_fit(
  model = "MVRSN", X = x_mvrsn,
  normalize_Psi = TRUE, precision = 1e-6, max_iter = 2
)
fit_mvrsn_custom <- mv_fit(
  model = "MVRSN", X = x_mvrsn,
  normalize_Psi = FALSE,
  M_init = M, A_init = A,
  Sigma_init = Sigma, Psi_init = Psi,
  eig_floor = 1e-8, monotone_tol = 1e-7,
  verbose = FALSE, precision = 1e-6, max_iter = 2
)

## 7. MVREN: custom initialization and all Q-matrix policies.
x_mvren <- mv_random(
  "MVREN", n, M, A = A, Sigma = Sigma, Psi = Psi,
  lambda = rep(1, p)
)
fit_mvren_warn <- mv_fit(
  model = "MVREN", X = x_mvren,
  M_init = M, A_init = A,
  Sigma_init = Sigma, Psi_init = Psi,
  q_policy = "warn", verbose = FALSE,
  precision = 1e-6, max_iter = 2
)
fit_mvren_strict <- mv_fit(
  model = "MVREN", X = x_mvren,
  q_policy = "strict", precision = 1e-6, max_iter = 1
)
fit_mvren_regularized <- mv_fit(
  model = "MVREN", X = x_mvren,
  q_policy = "regularize", precision = 1e-6, max_iter = 1
)

## Optional MVREN progress callback.
quiet_progress <- function(iteration, max_iter, criterion) invisible(NULL)
fit_mvren_callback <- mv_fit(
  model = "MVREN", X = x_mvren,
  q_policy = "warn", progress_callback = quiet_progress,
  max_iter = 1
)

```

Description

Runs repeated generate-and-fit experiments for a registered estimable model. This is intended to reproduce the Monte Carlo style used in the accompanying matrix-variate articles: data are generated from a known matrix-valued truth, the corresponding EM-type estimator is fitted, and estimation error is summarized across replications. Independent replications are distributed across all available logical CPU threads by default. ECM iterations remain sequential, preserving their mathematical update order. Random streams are reproducible across worker counts for a fixed seed.

Usage

```
mv_monte_carlo(
  model,
  sample_sizes = c(50, 100, 200, 400),
  replications = 200,
  truth,
  workers = NULL,
  precision = 1e-06,
  max_iter = 500L,
  seed = 123L,
  progress_dir = NULL,
  cens = NULL,
  Ind = 1L,
  verbose = FALSE,
  nu = NULL,
  lambda = NULL,
  epsilon = NULL,
  get.nu = NULL,
  nu_bounds = NULL,
  normalize_Psi = NULL,
  M_init = NULL,
  A_init = NULL,
  Sigma_init = NULL,
  Psi_init = NULL,
  q_policy = NULL,
  eig_floor = NULL,
  monotone_tol = NULL
)
```

Arguments

model	One of MVN, MVNC, MVSN, MVSNC, MVST, MVRSN, or MVREN.
sample_sizes	Positive integer vector of sample sizes.
replications	Positive number of replications per sample size.
truth	Named list containing the data-generating parameters, typically M, Sigma, Psi, and, for skewed or latent-effect models, A. MVST may also obtain nu from this list and MVREN may obtain lambda from it.
workers	Number of parallel workers; NULL uses all logical CPUs.

precision	ECM convergence tolerance.
max_iter	Maximum ECM iterations.
seed	Integer random seed.
progress_dir	Optional directory used to publish transient per-worker MVREN ECM iteration progress. Files are removed when each replication finishes. NULL disables progress reporting.
cens	Censoring proportion for MVNC and MVSNC. It must be supplied for censored campaigns.
Ind	Integer censoring/missingness mechanism for MVNC and MVSNC: 1 for interval censoring, 2 for missing values, and 3 for a mixture.
verbose	Logical verbosity control passed to models that support it.
nu	Optional degrees of freedom for MVST. If NULL, the value in truth\$nu is used.
lambda	Optional exponential-rate vector used when generating MVREN samples. If NULL, truth\$lambda is used; if that is also NULL, the MVREN generator uses its identified vector of ones.
epsilon	Optional numerical tolerance for MVRSN, MVSNC, and MVST.
get.nu	Optional logical control for updating nu in MVST.
nu_bounds	Optional length-two interval for the MVST nu update.
normalize_Psi	Optional logical control for MVRSN normalization of Psi.
M_init	Optional initial location matrix for MVRSN and MVREN.
A_init	Optional initial latent-effect/skewness matrix for MVRSN and MVREN.
Sigma_init	Optional initial row covariance matrix for MVRSN and MVREN.
Psi_init	Optional initial column covariance matrix for MVRSN and MVREN.
q_policy	Optional MVREN Q-matrix handling policy: "warn", "strict", or "regularize".
eig_floor	Optional positive eigenvalue floor for MVRSN.
monotone_tol	Optional MVRSN likelihood-monotonicity tolerance.

Value

A list with per-replication results, grouped summary, and the supplied truth. The summary contains one median_err_* column for each error measure present in results; when no valid estimate is available for a measure within a sample-size group, its median is returned as NA rather than omitting the column. The summary is descriptive and does not change model defaults or fitted estimates.

See Also

Other MVCens main interface: [mv_fit\(\)](#), [mv_random\(\)](#)

Examples

Each campaign below uses one replication and one ECM iteration only to
demonstrate the interface. Increase both values for a real simulation.

```
p <- 2
q <- 2
M <- matrix(0, p, q)
A <- matrix(c(0.4, 0.2, 0.1, 0.3), p, q)
Sigma <- matrix(c(1.0, 0.2, 0.2, 1.0), p, p)
Psi <- matrix(c(1.0, 0.1, 0.1, 1.0), q, q)
truth_mvn <- list(M = M, Sigma = Sigma, Psi = Psi)
truth_skew <- list(M = M, A = A, Sigma = Sigma, Psi = Psi)

## 1. MVN: one or several sample sizes can be studied.
mc_mvn <- mv_monte_carlo(
  model = "MVN", sample_sizes = c(20, 30), replications = 1,
  truth = truth_mvn, workers = 1,
  precision = 1e-6, max_iter = 1, seed = 123
)
mc_mvn$summary

## 2. MVNC: all three censoring/missingness mechanisms.
mc_mvnc_interval <- mv_monte_carlo(
  "MVNC", sample_sizes = 20, replications = 1, truth = truth_mvn,
  cens = 0.15, Ind = 1, workers = 1, max_iter = 1, seed = 123
)
mc_mvnc_missing <- mv_monte_carlo(
  "MVNC", sample_sizes = 20, replications = 1, truth = truth_mvn,
  cens = 0.15, Ind = 2, workers = 1, max_iter = 1, seed = 123
)
mc_mvnc_mixed <- mv_monte_carlo(
  "MVNC", sample_sizes = 20, replications = 1, truth = truth_mvn,
  cens = 0.15, Ind = 3, workers = 1, max_iter = 1, seed = 123
)

## 3. MVSNC: epsilon is forwarded to the skew-normal ECM fit.
mc_mvsn <- mv_monte_carlo(
  "MVSNC", sample_sizes = 20, replications = 1, truth = truth_skew,
  epsilon = 1e-8, workers = 1, max_iter = 1, seed = 123
)

## 4. MVSNC: all three mechanisms are supported here as well.
mc_mvsn_interval <- mv_monte_carlo(
  "MVSNC", sample_sizes = 20, replications = 1, truth = truth_skew,
  cens = 0.15, Ind = 1, epsilon = 1e-8,
  workers = 1, max_iter = 1, seed = 123
)
mc_mvsn_missing <- mv_monte_carlo(
  "MVSNC", sample_sizes = 20, replications = 1, truth = truth_skew,
  cens = 0.15, Ind = 2, epsilon = 1e-8,
  workers = 1, max_iter = 1, seed = 123
)
```

```

mc_mvsnMixed <- mv_monte_carlo(
  "MVSNC", sample_sizes = 20, replications = 1, truth = truth_skew,
  cens = 0.15, Ind = 3, epsilon = 1e-8,
  workers = 1, max_iter = 1, seed = 123
)

## 5. MVST: nu can come from truth or be supplied directly.
truth_mvst <- c(truth_skew, list(nu = 6))
mc_mvst_fixed <- mv_monte_carlo(
  "MVST", sample_sizes = 20, replications = 1, truth = truth_mvst,
  get.nu = FALSE, epsilon = 1e-8,
  workers = 1, max_iter = 1, seed = 123
)
mc_mvst_estimated <- mv_monte_carlo(
  "MVST", sample_sizes = 20, replications = 1, truth = truth_mvst,
  get.nu = TRUE, nu_bounds = c(2.01, 50), epsilon = 1e-8,
  workers = 1, max_iter = 1, seed = 123
)
mc_mvst_nu_argument <- mv_monte_carlo(
  "MVST", sample_sizes = 20, replications = 1, truth = truth_skew,
  nu = 6, get.nu = FALSE,
  workers = 1, max_iter = 1, seed = 123
)

## 6. MVRSN: Psi normalization can be enabled or disabled.
mc_mvrsn_normalized <- mv_monte_carlo(
  "MVRSN", sample_sizes = 20, replications = 1, truth = truth_skew,
  normalize_Psi = TRUE, eig_floor = 1e-8, monotone_tol = 1e-7,
  workers = 1, max_iter = 1, seed = 123
)
mc_mvrsn_custom <- mv_monte_carlo(
  "MVRSN", sample_sizes = 20, replications = 1, truth = truth_skew,
  normalize_Psi = FALSE,
  M_init = M, A_init = A, Sigma_init = Sigma, Psi_init = Psi,
  verbose = FALSE, workers = 1, max_iter = 1, seed = 123
)

## 7. MVREN: lambda is fixed to one under the identified parameterization.
truth_mvren <- c(truth_skew, list(lambda = rep(1, p)))
mc_mvren_warn <- mv_monte_carlo(
  "MVREN", sample_sizes = 20, replications = 1, truth = truth_mvren,
  q_policy = "warn", workers = 1, max_iter = 1, seed = 123
)
mc_mvren_strict <- mv_monte_carlo(
  "MVREN", sample_sizes = 20, replications = 1, truth = truth_mvren,
  q_policy = "strict", workers = 1, max_iter = 1, seed = 123
)
mc_mvren_regularized <- mv_monte_carlo(
  "MVREN", sample_sizes = 20, replications = 1, truth = truth_skew,
  lambda = rep(1, p), q_policy = "regularize",
  M_init = M, A_init = A, Sigma_init = Sigma, Psi_init = Psi,
  workers = 1, max_iter = 1, seed = 123
)

```

```
## Optional MVREN transient progress files.
mc_mvren_progress <- mv_monte_carlo(
  "MVREN", sample_sizes = 20, replications = 1, truth = truth_mvren,
  q_policy = "warn", progress_dir = tempdir(),
  workers = 1, max_iter = 1, seed = 123
)
```

mv_random

Generate observations from a registered matrix-variate model

Description

Validates the common matrix-variate generation contract and dispatches to the generator registered for model. The supported models follow the stochastic constructions used in the accompanying articles: matrix-normal row/column covariance for MVN, censoring and missingness for MVNC, half-normal skewing for MVSN and MVSNC, a Gamma scale mixture for MVST, row-specific half-normal effects for MVRSN, and row-specific exponential effects for MVREN. All generated observations use the same p by q shape as M ; Σ is the p by p row covariance or scale component and Ψ is the q by q column component. The model name is case-insensitive.

Usage

```
mv_random(
  model,
  n,
  M,
  A = NULL,
  Sigma,
  Psi,
  cens = NULL,
  Ind = 1L,
  nu = NULL,
  lambda = NULL,
  gamma_tilde = 2,
  rate = 1,
  return_latent = FALSE
)
```

Arguments

model	One registered model name. Matching is case-insensitive.
n	Positive integer giving the number of matrix observations.
M	Numeric p by q location matrix.

A	Optional p by q skewness or latent-effect matrix. It is required by MVSN, MVSNC, MVST, MVRSN, MVREN, MVNIG, and MVVG; it is ignored by MVN and MVNC.
Sigma	Positive-definite p by p row covariance or scale matrix.
Psi	Positive-definite q by q column covariance or scale matrix.
cens	Censoring proportion in $[0, 1]$. Required for MVNC and MVSNC; for MVNIG, NULL is interpreted as 0.
Ind	Integer censoring/missingness mechanism used by MVNC, MVSNC, and MVNIG: 1 for interval censoring, 2 for missing values, and 3 for a mixture of interval censoring and missingness.
nu	Degrees of freedom for MVST. This argument is required when generating from MVST.
lambda	Optional row-specific exponential rate vector for MVREN. Under the package identification convention, all supplied entries must be equal to one. NULL uses a vector of ones.
gamma_tilde	Positive inverse-Gaussian parameter used by MVNIG.
rate	Positive Gamma rate parameter used by MVVG.
return_latent	Logical. For MVRSN and MVREN, if TRUE, return both the generated array X and the latent variables W; otherwise return only the generated array.

Details

The following models are available: MVN, MVNC, MVSN, MVSNC, MVST, MVRSN, MVREN, MVNIG, and MVVG. MVNC, MVSNC, and MVNIG can return censored-data structures; MVVG uses rate; MVST uses nu; and the high-level MVREN interface uses the identified convention $\lambda_i = 1$.

Random-number generation is delegated to the model specification; this function does not fit parameters or alter the supplied scientific model. Set the R random seed before calling it when reproducibility is required. The returned object should be treated as generated data, not as evidence that a model has been fitted or validated for a real data set.

Value

The return type depends on the model and generation options:

- MVN, MVSN, MVST, and MVVG return a numeric array with dimensions p by q by n.
- MVRSN and MVREN also return that array by default; with `return_latent = TRUE`, they return a list with components X (the generated array) and W (the latent variables).
- MVNC and MVSNC return a list with X, cens, cc, and LS.
- MVNIG returns a list with X, cens, cc, LS, and X.or.

In particular, an ordinary MVN call returns the array itself, so it should be used as x, not x\$X.

See Also

[mv_fit\(\)](#), [mv_monte_carlo\(\)](#)

Other MVCens main interface: [mv_fit\(\)](#), [mv_monte_carlo\(\)](#)

Examples

```
## Common parameter matrices used throughout the examples
set.seed(123)
p <- 2
q <- 2
M <- matrix(0, p, q)
A <- matrix(c(0.4, 0.2, 0.1, 0.3), p, q)
Sigma <- matrix(c(1.0, 0.2, 0.2, 1.0), p, p)
Psi <- matrix(c(1.0, 0.1, 0.1, 1.0), q, q)

## 1. MVN: complete matrix-normal data
x_mvn <- mv_random(
  model = "MVN", n = 5, M = M,
  Sigma = Sigma, Psi = Psi
)
dim(x_mvn)
# MVN returns the array itself, not a list: use x_mvn, not x_mvn$X.

## 2. MVNC: matrix-normal data with the three supported mechanisms
x_mvnc_interval <- mv_random(
  model = "MVNC", n = 5, M = M,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 1
)
x_mvnc_missing <- mv_random(
  model = "MVNC", n = 5, M = M,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 2
)
x_mvnc_mixed <- mv_random(
  model = "MVNC", n = 5, M = M,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 3
)
names(x_mvnc_interval)

## 3. MVSN: complete matrix-variate skew-normal data
x_mvsn <- mv_random(
  model = "MVSN", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi
)
dim(x_mvsn)

## 4. MVSNC: skew-normal data with each censoring/missingness mechanism
x_mvsn_interval <- mv_random(
  model = "MVSNC", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 1
)
x_mvsn_missing <- mv_random(
  model = "MVSNC", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 2
)
x_mvsn_mixed <- mv_random(
  model = "MVSNC", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 3
)
```

```

)
names(x_mvsn_interval)

## 5. MVST: matrix-variate skew-t data; nu is required
x_mvst <- mv_random(
  model = "MVST", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, nu = 6
)
dim(x_mvst)

## 6. MVRSN: row skew-normal data
x_mvrsn <- mv_random(
  model = "MVRSN", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi
)
mvrsn_latent <- mv_random(
  model = "MVRSN", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, return_latent = TRUE
)
names(mvrsn_latent)
dim(mvrsn_latent$X)
dim(mvrsn_latent$W)

## 7. MVREN: row exponential-normal data
## The identified high-level interface uses lambda_i = 1.
x_mvren <- mv_random(
  model = "MVREN", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi
)
mvren_latent <- mv_random(
  model = "MVREN", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, lambda = rep(1, p),
  return_latent = TRUE
)
names(mvren_latent)
dim(mvren_latent$X)
dim(mvren_latent$W)

## 8. MVNIG: complete data (cens = NULL or 0) and censored data
x_mvsnig_complete <- mv_random(
  model = "MVNIG", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, gamma_tilde = 2
)
x_mvsnig_interval <- mv_random(
  model = "MVNIG", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 1,
  gamma_tilde = 2
)
x_mvsnig_missing <- mv_random(
  model = "MVNIG", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 2,
  gamma_tilde = 2
)

```

```

x_mvnmig_mixed <- mv_random(
  model = "MVNIG", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, cens = 0.20, Ind = 3,
  gamma_tilde = 2
)
names(x_mvnmig_complete)

## 9. MVVG: matrix-variate variance-gamma data
x_mvvg <- mv_random(
  model = "MVVG", n = 5, M = M, A = A,
  Sigma = Sigma, Psi = Psi, rate = 1.5
)
dim(x_mvvg)

```

prepare_beijing_mvren *Prepare Beijing Air Quality Data for Matrix-Variate Models*

Description

Download the Beijing Multi-Site Air Quality dataset from the UCI Machine Learning Repository and organize hourly measurements as a sample of 3×4 matrices. The function returns both original-scale and standardized observations for matrix-variate analysis.

Usage

```
prepare_beijing_mvren(output_dir = getwd())
```

Arguments

output_dir	Directory for the downloaded source files and processed results. Defaults to getwd(). The directory is created if needed. Supply a single, non-empty character string identifying a writable path.
------------	--

Details

Each observation represents one monitoring station on one calendar day. Matrix rows correspond to PM2.5, NO2, and O3, while columns correspond to four six-hour periods: 00:00–05:59, 06:00–11:59, 12:00–17:59, and 18:00–23:59. Observations are arranged by station and date.

Value

A named list containing:

X_raw Numeric $3 \times 4 \times n$ array of six-hour mean concentrations on the original scale.

X_std Numeric array of the same dimensions, with pollutant-wise standardized concentrations. Intended for model fitting.

obs_info Table linking each array slice to a station, date, observation index, and identifier.

`block_data` Long-form table of accepted six-hour means, observed hourly counts, and retained station-day identifiers.

`block_data_std` The same long-form data, including standardization constants and standardized concentrations.

`scaling` Pollutant-specific means (center) and sample standard deviations (scale).

`monthly_means` Monthly pollutant means computed from accepted six-hour blocks on the original concentration scale.

`pollutants` Names of the array rows.

`periods` Names of the array columns.

Data preparation

1. **Six-hour aggregation.** For each station, day, pollutant, and period, compute the mean concentration only when at least four hourly measurements are available.
2. **Completeness filter.** Retain a station-day only when all 12 pollutant-by-period means are available. Each retained station-day becomes one matrix in the array.
3. **Pollutant-wise standardization.** For each pollutant, calculate the mean and sample standard deviation across all periods and retained station-days. Use these values to standardize that pollutant's entries throughout the sample.

Thus, `X_std` is centered and scaled separately for each pollutant, pooling its entries over all periods and observations.

Download and saved files

Source archives and extracted CSV files are stored under the `beijing_air_quality_uci` subdirectory of `output_dir`. The archive is downloaded if it is not already present; extraction and preprocessing occur at every call. An initial download requires internet access.

Two processed files are saved directly in `output_dir`, overwriting existing files with the same names:

`beijing_mvren_processed.rds` The complete returned list; restore it using `readRDS()`.

`beijing_mvren_processed.RData` Separate objects `X_raw`, `X_std`, `obs_info`, `complete_blocks`, `complete_blocks_std`, `scaling`, and `monthly_means`; restore them using `load()`.

Notes

The function only prepares data; model estimation must be performed separately, for example with `mv_fit()`. The returned arrays contain no missing values after filtering. A diagnostic compares the retained sample size with the reference value of 16,133 and reports a warning if they differ. Successful download and extraction depend on the external repository and its archive structure.

See Also

[mv_fit](#), [readRDS](#)

Examples

```
## Not run:
# Download and process the data
beijing <- prepare_beijing_mvren(output_dir = "~/Beijing")

# Access the standardized array and observation metadata
X <- beijing$X_std
dim(X)
head(beijing$obs_info)
beijing$scaling

# Optionally fit a matrix-variate model
fit <- mv_fit(model = "MVREN", X = X,
              precision = 1e-6, max_iter = 200)

# Read saved results without preprocessing again
saved <- readRDS("~/Beijing/beijing_mvren_processed.rds")

## End(Not run)
```

```
prepare_dj_data
```

```
Prepare Quarterly Dow-Jones Data (1920-1934)
```

Description

Provides quarterly Dow-Jones dividends and divisor observations from 1920 through 1934 as a matrix-valued dataset. The observations are embedded in the function; no download is required.

Usage

```
prepare_dj_data(output_dir = getwd(), standardize = TRUE)
```

Arguments

output_dir	Directory in which to save the processed dataset. Defaults to the current working directory (getwd()).
standardize	Logical. If TRUE (default), also return an array standardized separately for each variable across all quarters and years.

Details

The data consist of 15 annual matrices of dimension 4×2 . Rows identify quarters Q1-Q4; columns contain DJ dividends and DJ divisor. The annual matrices are stacked into a $4 \times 2 \times 15$ array, with years ordered chronologically. Standardization uses the pooled mean and sample standard deviation of each column variable.

Value

A list with three components:

`X_raw` Original data as a $4 \times 2 \times 15$ array.

`X_std` Variable-wise standardized array, or NULL when `standardize = FALSE`.

`dj_data` Named list of the 15 original annual matrices.

Saved file

Saves `dow_jones_1920_1934_processed.rds` in `output_dir`, overwriting an existing file of that name. Reload with `readRDS()`.

Note

Annual observations follow consecutive years and should not automatically be assumed independent in statistical applications.

See Also

[mv_fit](#), [readRDS](#)

Examples

```
## Not run:
dj <- prepare_dj_data(output_dir = "~/DowJones")
dim(dj$X_raw)
dj$dj_data[["1920"]]
dim(dj$X_std)

## End(Not run)
```

Index

* MVCens MVREN functions

dmvren, 3
loglik_mvren, 10
mvren_article_parameters, 15
mvren_covariances, 16
mvren_mean, 17

* MVCens MVRSN functions

dmvrsn, 5
loglik_mvrsn, 11
mvrsn_article_parameters, 18
mvrsn_covariances, 19
mvrsn_mean, 20

* MVCens density functions

dmvren, 3
dmvrsn, 5
dmvsn, 6
dmvst, 7

* MVCens likelihood functions

loglik_mvn, 8
loglik_mvnc, 9
loglik_mvren, 10
loglik_mvrsn, 11
loglik_mvsn, 12
loglik_mvsn, 13
loglik_mvst, 14

* MVCens main interface

mv_fit, 20
mv_monte_carlo, 25
mv_random, 30

dmvren, 3
dmvren(), 5–7, 10, 11, 15, 17, 18
dmvrsn, 5
dmvrsn(), 4, 6, 7, 12, 18–20
dmvsn, 6
dmvsn(), 4, 5, 7
dmvst, 7
dmvst(), 4–6, 14

loglik_mvn, 8

loglik_mvn(), 9, 11–15
loglik_mvnc, 9
loglik_mvnc(), 8, 11–15
loglik_mvren, 10
loglik_mvren(), 4, 8, 9, 12–15, 17, 18
loglik_mvrsn, 11
loglik_mvrsn(), 5, 8, 9, 11, 13–15, 18–20
loglik_mvsn, 12
loglik_mvsn(), 8, 9, 11, 12, 14, 15
loglik_mvsn, 13
loglik_mvsn(), 8, 9, 11–13, 15
loglik_mvst, 14
loglik_mvst(), 7–9, 11–14

mv_fit, 20, 35, 37
mv_fit(), 2, 3, 27, 31
mv_monte_carlo, 25
mv_monte_carlo(), 2, 3, 22, 31
mv_random, 30
mv_random(), 2, 3, 22, 27
MVCens (MVCens-package), 2
MVCens-package, 2
mvren_article_parameters, 15
mvren_article_parameters(), 4, 11, 17, 18
mvren_covariances, 16
mvren_covariances(), 4, 11, 15, 18
mvren_mean, 17
mvren_mean(), 4, 11, 15, 17
mvrsn_article_parameters, 18
mvrsn_article_parameters(), 5, 12, 19, 20
mvrsn_covariances, 19
mvrsn_covariances(), 5, 12, 18, 20
mvrsn_mean, 20
mvrsn_mean(), 5, 12, 18, 19

prepare_beijing_mvren, 34
prepare_dj_data, 36

readRDS, 35, 37